Frontier Models as Weight Editors for Gemma 4 E4B

Ozgun Genc
ozgungenc@gmail.com

A gradient-free weight edit cuts a coding agent's escape-code errors from 15 % to 3 % in live runs, and shows what such fixes must also preserve

Extended version of our Kaggle Writeup for the Gemma 4 Developer Agent Competition, with the result tables, the first round, the editors' decisions and the retry checks. Protocols, run records and audits are in the repository.

Abstract

We study whether frontier models can change a coding agent's behaviour by editing its weights directly. The target is one defect of Gemma 4 E4B: when it edits a file, it often copies escape codes such as `\n` into the text it means to replace. Editors could read the weights and send at most 25 requests to run the model, each regenerating up to 128 recorded calls or reading activations on up to 32 texts, but could not compute gradients. In a test fixed in advance on 320 recorded edit calls from 69 held-out repositories, three submitted edits cut these escape artefacts by 23, 53 and 75 %, beyond random edits of the same size, but each also lost 3 to 11 of 213 previously working edit calls in its worse generation run, where our preservation rule allowed 2 per run. Served as a LoRA adapter on 4-bit E4B in vLLM, one selected edit cut the defect in live agent runs on 148 new repositories from 15 % to 3 % of edit calls, and from 7 % to 1 % with thinking (reasoning before each action) on, in two replications planned before their runs, and it ran without errors in the competition's own scorer. Gradient training on the same calls removed more artefacts and failed the same rule. We detected no difference in solved tasks: the harness already repairs many escaped copies. Re-scored afterwards without that repair, the base model has fewer working calls, but two of the three edits lose none of these and raise the share of old texts found by 4 and 7 points. Most of the base model's misses are retries after an earlier miss, and two checks fixed in advance found little room for an edit there. We release the library, a new task set and the replay test set.

1 Introduction

Gemma 4 E4B is a small open model (about 4.5 billion effective parameters, 42 layers). In the Gemma 4 Developer Agent Competition on Kaggle, it works as a coding agent inside the competition's harness, which gives it tools to read and edit the files of a Python repository and scores it by the share of hidden bug-fix tasks it resolves. To change a file, the agent calls an edit tool with the *old text*, copied exactly from the file, and the new text. Tool results reach the model as escaped strings, because the agent framework encodes each result as JSON text before the chat template shows it: a line break becomes the two characters `\n` and a quote becomes `\"`. The model often copies that form into the old text (Figure 1). We call each such leftover an *escape artefact*. A generated text counts as having one when it contains `\n` or `\"` as two characters and the file being edited contains no such sequence.

Preprint.



Figure 1: The defect, in a synthetic example: the unedited model copies escape codes into the old text; the edited model writes the real characters.

Failed edits are frequent: in 412 agent runs on the competition's four training repositories, 405 of 1,191 edit calls failed because the old text was not in the file. The harness already softens the escape defect. When an old text is not found, a fallback built into the edit tool unescapes both the old text and its replacement and tries again, in every agent run and in the scorer; in those runs it rescued 108 of 614 accepted edits. So we did not expect fewer escape artefacts to solve many more tasks. We chose the defect because it is frequent, countable and specific, which makes it a clean test of a narrower question: can a frontier model deliberately change one behaviour of a small model by writing its weights, without breaking the calls that already work?

Frontier models have recently been shown to write changes to neural-network weights directly, an approach called weight-space programming [1]. We test the idea on one behaviour of a software-engineering agent, along three separate questions (Figure 2): does the behaviour change, are working calls preserved, and does the agent solve more tasks? We contribute a controlled experiment with frontier-model weight editors, held-out repositories and size-matched random controls; tests of one selected edit in live, quantized serving and against gradient training on the same calls; and a library, a task set and a replay set for repeating such tests. The experiment tests particular edits under fixed budgets; it does not map the limits of weight editing.

2 Method

2.1 Editors

The study ran in two rounds. This paper reports mainly the second, aimed at escape artefacts; the first, aimed at the broader failure of an old text not found at all, is summarised in Section 3.5. Each second-round editor worked alone in a sandbox, with the weights and recorded calls from the competition's four training repositories (fastapi, rich, requests and httpx) but no access to the test data (Figure 3a). It could read any weight and spend a budget of *forward-pass requests* on a server that held the model. One request either regenerated the old text on up to 128 recorded calls, with or without the editor's current edit, and returned the share of old texts found (acceptance) and the artefact rate, or ran up to 32 texts through the model and returned log-probabilities and activations. Claude Opus 5.5 and Claude Fable 5.1 each had 25 requests and GPT-6 Astra had 10. The edits are *gradient-free*: backpropagation and iterative optimisers were not allowed, but closed-form fits on activations, such as least squares, were. No editor's

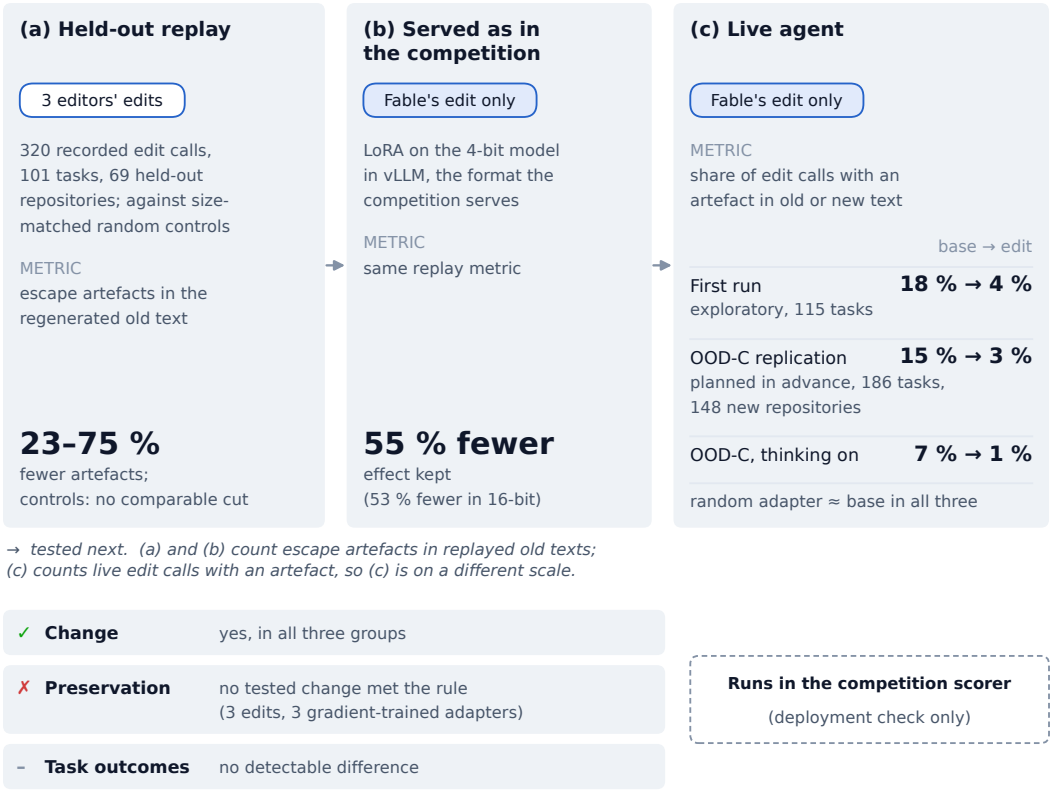


Figure 2: Evidence map: three groups of tests, each with its own metric and edits; arrows mean "tested next". (a) The replay test of Section 2: three editors' edits on 320 recorded calls, against random controls of the same size. (b) Fable's edit (by Claude Fable 5.1) served as a LoRA adapter on the 4-bit model, the format the competition serves, on the same replay (55 % fewer artefacts; 53 % in 16-bit). (c) Live agent runs (Section 3.3): an exploratory first run and two replications planned in advance on OOD-C, a new set of 186 tasks, with thinking off and on. Below: the three outcomes, over three edits and three gradient-trained adapters (Section 3.2), and a deployment check in the competition's scorer.

scripts or transcripts contain a gradient or optimiser call. Each editor could submit one edit or none (Table 1). Two later *informed* runs, by Opus and by Claude Fable 5.2, could read the three earlier editors' notebooks, answers and edits; each was registered separately, that is, planned in advance with its own time-stamped plan in the study's log (Section 2.2). A second Fable 5.2 run tried to repair a gradient-trained adapter (Section 3.2). These three later runs had 10 requests each.

Table 1: Second-round editor arms. "Informed" arms could read the earlier editors' work; each was registered separately.

Arm	Model	Forward-pass budget	Started from	Submitted
Opus	Claude Opus 5.5	25	base model	an edit
Fable	Claude Fable 5.1	25	base model	an edit
Astra	GPT-6 Astra	10	base model	nothing
Informed Opus	Claude Opus 5.5	10	base model, earlier editors' records	an edit

Arm	Model	Forward-pass budget	Started from	Submitted
Informed Fable 5.2	Claude Fable 5.2	10	base model, earlier editors' records	nothing (withdrew)
Repair Fable 5.2	Claude Fable 5.2	10	a gradient-trained adapter (Section 3.2)	nothing (withdrew)

Each submission is a set of low-rank changes: each changed weight matrix W becomes W + BA, where B and A are two thin matrices. This form is a LoRA adapter [2], which the competition's vLLM server can load. Fable's edit is a rank-2 change in a single matrix, the last layer's feed-forward output matrix. Its A matrix reads two detector values from the activations entering that matrix, one high just before an escaped line break and one just before an escaped quote, and its B matrix turns them into lower scores for the escaped tokens and higher scores for the raw characters; both were fitted with linear solves. Opus's edit instead changes several matrices: it turned 40 last-layer feed-forward neurons that it judged little used into detectors, switched on by a rank-1 flag written in layer 40 (layers are numbered 0 to 41, so this is the second-to-last). Informed Opus used 80 such neurons, and the withdrawn candidate of the informed Fable 5.2 run 86. These descriptions are the editors' own.

2.2 How we judged an edit

We judged each frozen edit by replay (Figure 3b). For a recorded edit call, the model regenerates only the old text from the exact original prompt, and the harness's matching code checks whether it is found in the file; a call whose old text is found is *accepted*. In a *fidelity check*, replaying the base model on the held-out calls described next reproduced 95 of the 103 calls that had failed when recorded and all 213 that had succeeded (the other 4 had failed for other reasons, 3 of them because the old text matched several places).

The test was fixed in advance [3], that is, written down with a time stamp in the study's log before any edit ran on its calls (see the end of this section); we call it the *gate*. It uses 320 calls from 101 tasks in 69 held-out repositories, which share no repository with the training repositories or the editors' data. Each call is generated in two *draws*, one greedy and one at temperature 0.6, so each model gives 640 generations. Each edit faced the base model and two random controls with the same location, rank and size (the Frobenius norm of the weight change) but a random direction. A task's artefact rate is the mean over its calls and both draws, and the edit's rate had to fall against all three references in one-sided exact sign tests over tasks, with a Holm correction across Opus's and Fable's edits; the separately registered informed Opus edit was tested on its own. Three guards limited side effects:

- *acceptance*, the share of old texts found, averaged over tasks, could fall by at most 1 point;
- among recorded calls that had succeeded and that the base model's replay also matched (213 per draw), which we call *working calls*, at most 2 could be lost in each draw (negative flips [12]; the *lost-call guard*); these are accepted old-text matches, not correct bug fixes;
- *compile failures*, the share of accepted calls whose edited Python file no longer compiles, could rise by at most 2 points. For this guard only, a second replay, the *full-call replay*, regenerates both the old and the new text, because the fallback changes both. This full-call form replaced the registered form, which paired the regenerated old text with the recorded new text, before any edit ran on the test calls. Table 2 gives both verdicts: the amendment turns all three submitted edits' compile verdicts from fail to pass but changes no overall verdict, because each edit also fails the lost-call guard.

Registered, throughout, means planned in advance: a time-stamped plan in the study's log before the run it governs. The round-2 rule was committed locally before the first forward pass and pushed to GitHub while the search was under way. The live replications and the training comparison each had a plan with an expected effect, frozen and hashed in our log before their

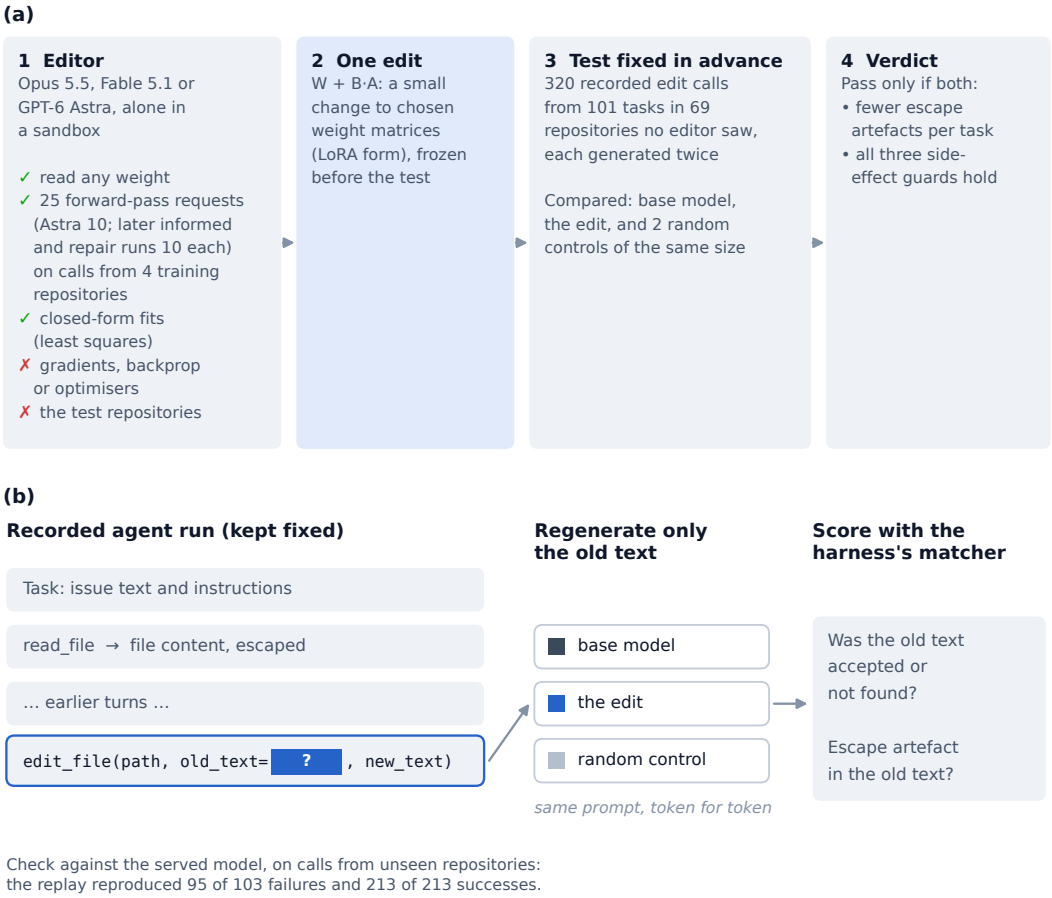


Figure 3: (a) What an editor could and could not do. (b) The replay regenerates only the old text; everything before it stays fixed. The fidelity check compares the base model's replay with the outcomes recorded from the served agent.

runs and committed to git afterwards. We chose the target after counting artefacts in the base model's test calls.

2.3 Task sets and the live metric

Our held-out task sets are named OOD (out of distribution), since their repositories are outside the competition's training repositories. The replay calls come from E4B agent runs on two of them: OOD-43, a public Kaggle set of 45 recent bug-fix tasks from 17 repositories, 43 of them gradable, and OOD-B, 82 tasks from 70 repositories that we drew from SWE-rebench [4]. The replay keeps the *usable* calls of five such runs, those with a verified prompt, a certain file state and an old text: 209 calls from four runs on OOD-43 and 111 from one run on OOD-B. The first live run used 115 of the two sets' tasks (44 and 71; the rest could not be run or graded on our Colab setup). For the planned replications we built OOD-C: 186 tasks from 148 repositories, filtered from SWE-rebench [4, 5], packaged to run offline in the competition harness, and checked by script to share no repository with training, the earlier sets or the replay calls. Only 53 OOD-C tasks meet our full gradability rule (the reference fix passes twice, the empty patch fails, and the reference fix passes again under Python 3.12). A task is *solved* when the agent's patch passes the task's hidden tests, as in SWE-bench, and we count solved tasks on these 53. So OOD-C mainly measures what the agent writes. In live runs the agent writes both texts itself, so we count edit calls whose old *or* new text carries an artefact; since that check needs the file's content at the

moment of the call, we leave out calls whose file state could not be rebuilt from the run's trace. Counting the old text alone gives the same picture.

3 Results

3.1 Targeted change and preservation

All three submitted second-round edits reduced the defect, and every one lost more working calls than the rule allowed (Figure 4, Table 2). Of the base model's 640 replay generations, 100 carried an artefact; Opus's edit left 77, Fable's 47 and informed Opus's 25: cuts of 23, 53 and 75 %, each with $p \leq 0.011$ against the base and both of its random controls. The six random controls left 99 to 102. But the three edits lost up to 3, 5 and 11 of the 213 working calls in a draw, against a limit of 2, and Fable's (narrowly, −1.01 points) and informed Opus's edits also failed the acceptance guard.

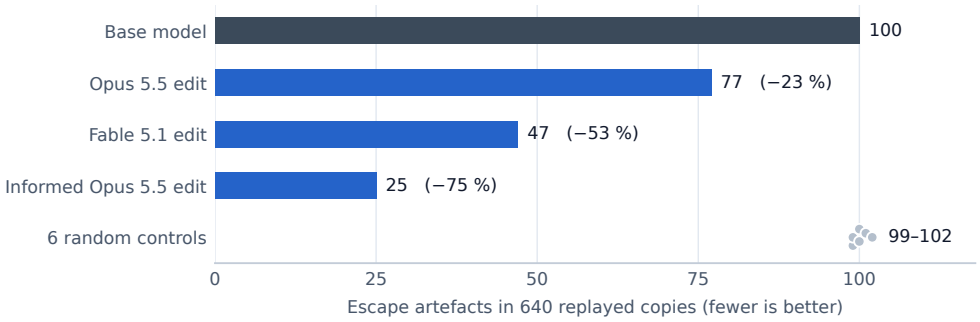


Figure 4: Every edit cut escape artefacts beyond its own two random controls. The edits differ in size, so each is compared with controls of its own location, rank and size (the Frobenius norm of the weight change).

Table 2: Second-round results: every edit passed the artefact test and failed at least one guard (320 calls, two draws each).

Edit (requests)	Escape artefacts of 640 ↓ (base 100)	Tasks fewer / more	Largest p of 3	Acceptance change, points (≥ -1)	Working calls lost per draw (≤ 2)	Compile failures, registered / full call ($\leq$ 11.0 / 9.9 %)	Passed
Opus 5.5 (25)	77	9 / 0	0.011	+0.4	2, 3	12.5 / 8.4 %	no
Fable 5.1 (25)	47	20 / 0	9.5×10^{-7}	−1.01	5, 5	15.3 / 9.1 %	no
Informed Opus 5.5 (10)	25	25 / 1	4.0×10^{-7}	−4.2	11, 11	15.9 / 8.4 %	no
Six random controls	99–102						

In a descriptive check after the gate, the activations show how narrow Fable's edit is (Figure 5). We ran 120 test calls once more (all 48 whose unedited old text has an artefact, and 72 others) with *teacher forcing*: instead of letting each model generate, we fed in the unedited model's own old text token by token and read, at each position, what the unedited and the changed models would predict next. We also recorded the activations entering the last layer's feed-forward output matrix, where the edit sits; at 99.94 % of positions the unedited model's recomputed most likely

next token equals the one it wrote. The edit's two rank components act as detectors: the first is high before a `\n` (median 0.65 at 107 positions), the second before a `\"` (median 0.78 at 196), and both stay near zero elsewhere (10th to 90th percentile −0.05 to +0.04). At 14,255 ordinary positions the edit changes the most likely next token twice, no more often than the two random controls (1 and 5), and writes a median 2 % of the feed-forward output's norm there. In 30 of the 48 texts with an artefact it changes the most likely token at the first position where an escape code was written. A gradient-trained change of the same matrix, rank and size (Section 3.2) changes the first escape in 36 texts, but writes a median 31 % of the output's norm at ordinary positions and changes the most likely token at 16 of them; that order matches the gate, where it left fewer artefacts than Fable's edit (23 against 47) and lost more working calls (12 and 13 against 5 and 5). Because the text follows the base model's escaped output, positions after a text's first escape sit in a context the edited model would not produce; the first-escape counts are the closest to generation. Along the unedited model's own text, then, the edit acts almost only where escape codes are written; the next paragraph shows how acting there can still lose working calls.

A cleaner-looking copy can be less reliable than an escaped one (Figure 6). When the base model wrote an escaped old text, the fallback often repaired it and the call succeeded. Writing raw text there gains nothing, and the raw copy is sometimes wrong, for example with a closing brace dropped (Figure 6b). Of the 84 escaped copies that the fallback had rescued for the base model, over both draws and all calls, the three edits made 24, 45 and 67 no longer need the fallback, most of them by writing exact copies, and 5, 12 and 24 of these moved copies were no longer found (Figure 6a). These counts include calls outside the working set, so they do not add up to the per-draw losses above. Average acceptance can hide such losses when other calls improve, so the rule counts them separately.

The same generations show what that repair hides. Scored again with the harness's own matching code but its unescape fallback switched off (post hoc; nothing is regenerated), the base model's accepted calls fall from 443 to 359 of 640, and its working calls per draw from 213 to 177 and 174. Opus's and Fable's edits then lose none of these, pass all three guards and raise acceptance by 4.1 and 7.3 points (better in 10 and 17 tasks, worse in none; also beyond both random controls in sign tests). Informed Opus's edit gains 9.3 points but misses the compile guard by 0.1 point. No agent has run without the fallback.

The editors' own decisions add context; they are single runs, not a ranking of models. Informed Opus named the lost-call guard as its main risk and submitted anyway: on the training-repository calls it had measured itself, its edit had lost up to 5 calls per draw, against 11 here, while its artefact cut transferred (82 % there, 75 % here). Astra submitted nothing, because its best candidate showed no effect on tasks it had set aside; it also found, on its own, the flaw in the registered compile guard, which the other two blind editors described too and our amendment then addressed. Both Fable 5.2 runs found effects but withdrew after measuring side effects themselves. The informed run's best candidate cut artefacts by two thirds on fresh search calls but broke legitimate escapes, such as `\n\n` in new code. The repair run found that the gradient-trained adapter it started from cut artefacts but ended quoted spans early, and that no repair kept as many accepted-and-compiling edits as the base model; on this harness, it argued, the fallback already accepts the artefact calls. Gated afterwards off the record, the informed run's withdrawn candidate left 43 artefacts against 100 but failed all three guards (acceptance −1.27 points, 3 working calls lost per draw, compile failures +3.7 points), close to its editor's own estimates.

3.2 Comparison with gradient training

Editing by reasoning would be most useful if it needed less data than training. In a comparison fixed in advance, we trained LoRA adapters with gradients to write the correct old text on calls from the training repositories. A call is *labelled* when its correct old text is known: the recorded old text or its unescaped form occurs exactly once in the file, which holds for 411 of the 730 calls the editors could search. The matched arm used the 39 labelled calls among those Fable had read, at Fable's location, rank and size. Two larger arms trained all linear layers, on the same 39 calls and on all 411 labelled calls; these two are not size-matched. Only the 39-call arms tuned their learning rate, on validation calls from the training repositories.

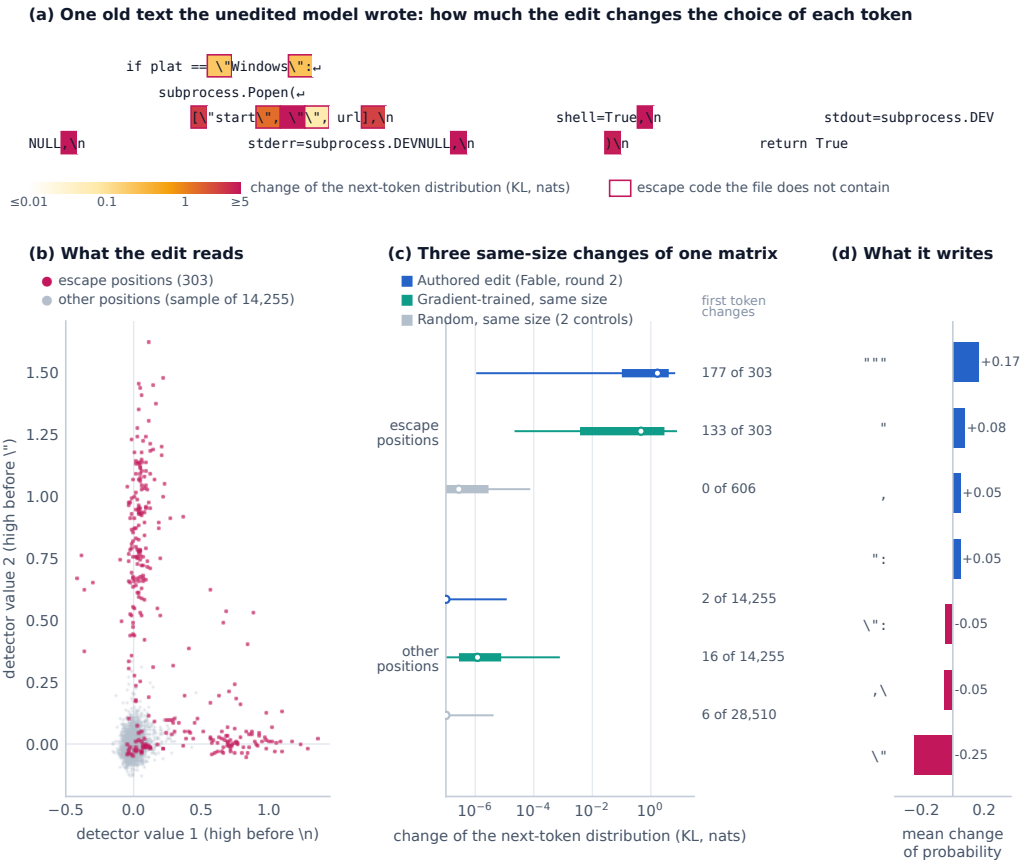


Figure 5: What Fable's edit does at each token, on 120 test calls (all 48 whose unedited old text has an artefact, and 72 others), teacher-forced along the unedited model's own old text; measured after the gate, descriptive. (a) One recorded old text, shaded by how much the edit changes the next-token distribution; a bent-arrow symbol marks a real line break the model wrote, other line breaks in the panel are display wrapping only, and boxed codes are escape codes the file does not contain. (b) The edit's two detector values, read from the activations entering the last layer's feed-forward output matrix: near zero at 14,255 ordinary positions; at the 303 positions where an escape code was written, the first is high before \n and the second before \". (c) Three kinds of change to that matrix with the same rank and size. Lines span the 5th to 95th percentile of the change, bars the middle half and dots the median; changes below 10^{-7} nats are drawn at the left edge, as hollow dots. The counts on the right are positions where the most likely next token changes ("first token changes"): 177 of the 303 escape positions under Fable's edit and 133 under the gradient-trained one; at ordinary positions 2 and 16, against 1 and 5 for the two random controls, whose counts are pooled. (d) Averaged over the 303 escape positions, the edit moves probability from escaped tokens such as \" (−0.25) to unescaped quote tokens, most of it to the single token """ (+0.17).

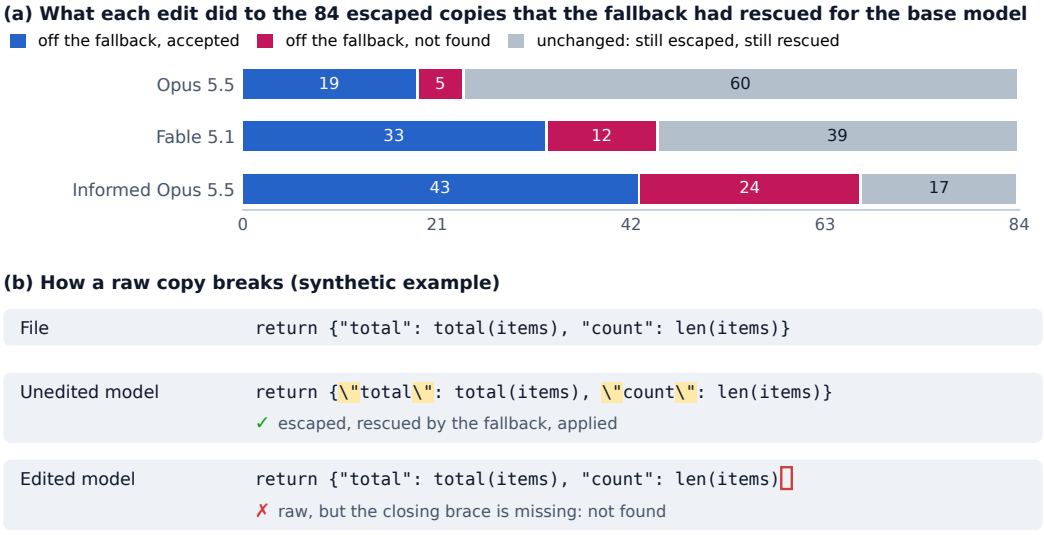


Figure 6: (a) What each edit did to the escaped copies that the fallback had rescued for the base model: some left the fallback and were still accepted, some were no longer found. (b) How a raw copy breaks (synthetic example).

Training removed more artefacts and failed the same rule (Figure 7, Table 3). At Fable's location the trained adapter left fewer artefacts than Fable's edit in 13 tasks and more in 1 ($p = 0.002$). Every arm lost more than 2 working calls per draw; the 411-call arm passed every other guard. It was also cheap: a 39-call run took about 1 h and the 411-call run 7.3 h on the same Mac, while Fable's 25 forward-pass requests took 10.6 h. On this target the authored edit showed no data advantage over training, and the loss of working calls was not specific to gradient-free editing. Nor must removing escape artefacts break calls: the all-layer arms removed more artefacts than the matched arm and lost fewer working calls (5 and 7 against 13 in the worse draw). Without the fallback (Section 3.1) the adapters gain 9 to 16 points of acceptance, though each still misses a guard: the matched adapter the lost-call and compile guards, the all-layer adapter trained on 39 calls the compile guard, and the one trained on 411 calls the lost-call guard.

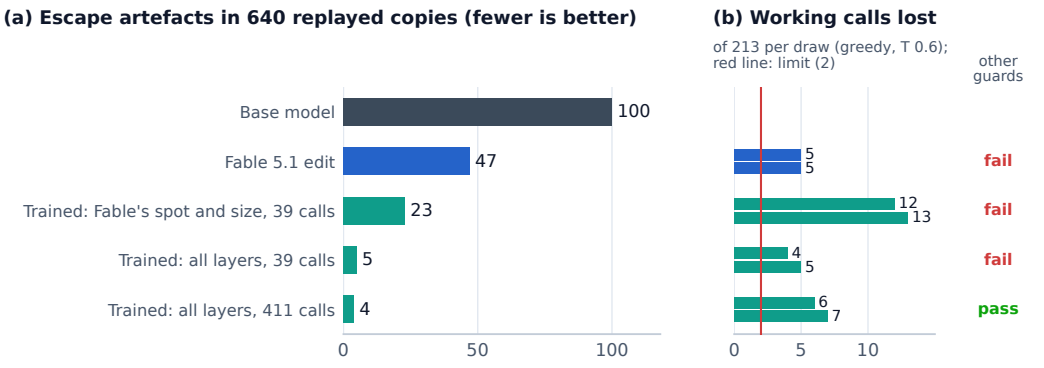


Figure 7: Gradient training on the same calls removed more artefacts than Fable's edit, and every arm lost more than 2 working calls per draw.

Table 3: Gradient-trained adapters against Fable's edit on the replay test (the same rule as Table 2).

Arm	Escape artefacts of 640 ↓ (base 100)	Acceptance change, points (≥ -1)	Working calls lost per draw (≤ 2)	Compile failures, full call (≤ 9.9 %)	Passed
Fable 5.1 edit (25 requests)	47	−1.0	5, 5	9.1 %	no
Trained at Fable's location and size, 39 calls	23	−3.6	12, 13	10.3 %	no
Trained on all layers, 39 calls	5	+1.1	4, 5	10.7 %	no
Trained on all layers, 411 calls	4	+0.3	6, 7	5.2 %	no

3.3 Transfer to the running agent

We chose Fable's edit after seeing the replay results, so this step is exploratory. Served as a LoRA adapter in vLLM on E4B with 4-bit weights and 16-bit activations (W4A16), as the competition serves it, the edit kept its effect on the replay: artefacts fell from the 4-bit base model's 102 to 46, close to its 16-bit result, while a random adapter left 100. The random adapter, used in all live runs below, is the gate's first random control for Fable's edit, served the same way. Its cost survived too: 7 and 11 working calls lost per draw, against the 4-bit base model's own replay (5 in each draw in 16-bit; none for the random adapter). In a rehearsal on the scorer's hardware vLLM served all 223 requests through the adapter without errors, and agents with this edit ran without errors in the competition's scorer, which runs submitted agents on hidden tasks. On its public leaderboard (about 58 tasks, so one task is about 0.017) they resolved 0.05 of tasks with thinking off and on, like our comparable E4B agents without the edit (0.05 to 0.08). The board shows only that the edit runs there, not what it changes.

In live agent runs the edit cut the defect again, on new repositories, under a plan fixed in advance (Figure 8, Table 4). A first, exploratory run on the 115 OOD-43 and OOD-B tasks gave 4 % of counted edit calls with an artefact, against 18 % for the base and 20 % for a random adapter. Because those repositories had supplied the replay calls, we froze the edit, the analysis, the pass rule (an artefact rate lower than both the base's and the random adapter's in one-sided sign tests over tasks, each $p < 0.05$) and an expected effect (the edit's rate at 0.2 to 0.6 times the base's), and repeated the three arms on OOD-C. Artefacts fell from 15 % of counted edit calls for the base to 3 % for the edit, with 16 % for the random adapter. Per task, the edit was lower than the base in 34 tasks and higher in 4 ($p = 3 \times 10^{-7}$), and lower than the random adapter in 27 and higher in 7 ($p = 4 \times 10^{-4}$; one-sided sign tests); a repository-level bootstrap also supported the cut. Live runs count artefacts only, so they leave in place the replay's finding that the edit loses working calls.

The agent we submit runs with *thinking* on, reasoning before each action; the runs above had it off. A second planned replication on the same OOD-C tasks, with thinking on, gave 7 % for the base, 1 % for the edit and 8 % for the random adapter. The edit was lower than the base in 16 tasks and higher in 4 ($p = 0.006$), and passed against the random adapter too ($p = 2 \times 10^{-4}$). The two replications were separate experiments, not a paired comparison of thinking off and on, so the gap between their base rates (15 % and 7 %) is not a measured effect of thinking. In all three runs the edit made the agent lean less on the fallback, which rescued 3 to 6 % of its accepted edits against 9 to 19 % for the base.

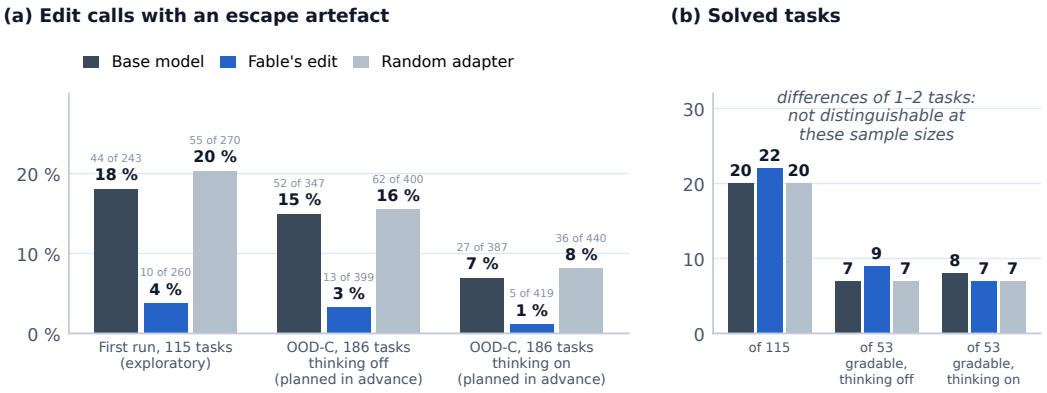


Figure 8: In live runs the selected edit cut escape artefacts with thinking off and on, and no difference in solved tasks was detected. Shares pool counted calls; the tests compare paired tasks. In (b), solved-task counts that differ by one or two tasks cannot be told apart with samples of this size, which does not show equivalence.

Table 4: Live agent runs: one run per arm, served as a LoRA adapter on the 4-bit model in vLLM.

Task set	Arm	Edit calls with an artefact ↓	Old text not found	Accepted only through the fallback	Solved tasks
First run: 115 tasks, OOD-43 and OOD-B (exploratory)	Base	18 % (44 of 243)	38 %	19 %	20 of 115
	Fable's edit	4 % (10 of 260)	35 %	6 %	22 of 115
	Random adapter	20 % (55 of 270)	34 %	24 %	20 of 115
OOD-C, 186 tasks, thinking off (planned in advance)	Base	15 % (52 of 347)	27 %	15 %	7 of 53
	Fable's edit	3 % (13 of 399)	34 %	5 %	9 of 53
	Random adapter	16 % (62 of 400)	32 %	14 %	7 of 53
OOD-C, 186 tasks, thinking on (planned in advance)	Base	7 % (27 of 387)	25 %	9 %	8 of 53
	Fable's edit	1 % (5 of 419)	35 %	3 %	7 of 53
	Random adapter	8 % (36 of 440)	36 %	8 %	7 of 53

3.4 Task outcomes and stuck retries

We detected no difference in solved tasks (Table 4). Comparisons this small cannot show equivalence. On OOD-C, the share of edit calls whose old text was not found rose with both adapters, the random one included (thinking off: 27 % for the base, 34 % for the edit, 32 % for the random adapter; thinking on: 25 %, 35 % and 36 %), so the rise is not specific to the edit's direction; its cause is untested.

Most of the base model's misses, the recorded edit calls in the held-out runs whose old text was not found, were retries: 131 of 202 came after an earlier miss in the same task, and 55 % of retries repeated, byte for byte, an old text that had already failed. Before planning a third round aimed at them, we fixed two go/no-go checks in advance. Sampling each of 71 held-out retries 8 times showed two kinds (Figure 9a): 35 repeated a failed old text in every draw and 32 never did, and sampling almost never moved a *stuck* call off its failed text. A linear probe told retries from first attempts well (AUC 0.96), so the state is readable. But even with every failed old text deleted from the model's own earlier turns, stuck calls repeated it in 48 % of draws and only 17 % were accepted, far below the 40 % bar we had set (Figure 9b). Both checks failed, so we did not run the third round.

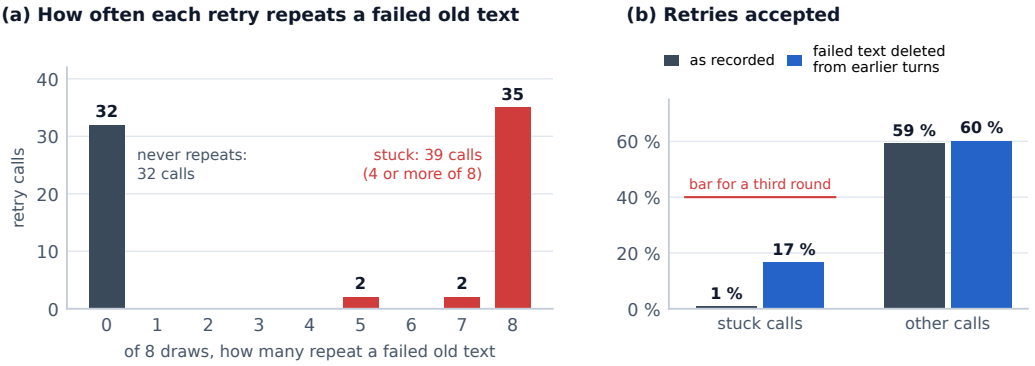


Figure 9: (a) Retries either always repeat a failed old text or never do. (b) Even with the failed text deleted from the context, few stuck retries are accepted; non-stuck acceptance stays near 60 %.

3.5 Round 1 and a tool-side change

A first round aimed at a broader target, whether the old text is found at all. Opus and Fable each wrote one edit with 10 forward-pass requests, and a recipe of ours a third. On 225 held-back calls from the training repositories none helped (264, 264 and 262 accepted of 450 generations, against 264 for the base), and size-matched random edits moved acceptance as much. The recipe removed a direction that predicted the failure (AUC 0.88), yet nothing changed; a probe's success does not show that the model uses what it reads [11]. Independent checks reproduced parts of the editors' explanations of their edits, but the effects were far too small to change what the model wrote.

A tool-side change did what the weight edits did not. In a test planned in advance on the same calls, showing tool results in the chat template's native form instead of an escaped string raised acceptance by 2.0 points ($p = 0.031$); its replay re-rendered each recorded prompt in that form. The effect is small and narrow: the change bundles escaping, field layout and slightly shorter prompts, and the calls had already served round 1's test. Round 1's edits left escape artefacts unchanged; that, with the artefact count in the base model's held-out calls, led to the narrower target of round 2.

We later ran the same tool-side change on the round-2 gate calls under a registration made before any generation, which fixed scoring under both rules, with and without the fallback; a tool-side change has no size-matched random control. It cut escape artefacts from 100 to 78 of 640 (fewer in 12 tasks, more in none), close to Opus's edit (77). With the fallback, as in the competition's harness, acceptance did not move (+0.1 points; better in 4 tasks, worse in 4), and 1 and 2 working calls were lost per draw, at the limit of 2. Without the unescape fallback, the second registered rule, acceptance rose by 3.7 points (better in 12 tasks, worse in 1) and none of the 177 and 174 working calls was lost. Of the escaped copies the fallback had rescued, the change made 21 no longer need the fallback and 4 of these were no longer found, against 24 and 5 for Opus's edit. In the full-call replay, which regenerates both texts, the change left 56 of the base's 89 escaped

old texts and 39 of its 67 escaped new texts (Opus's edit 65 and 47, Fable's 34 and 17), short of the halving we had set beforehand as the mark for the escaped view being their main source. There it also missed the compile guard, whose limit is the base's rate plus 2 points, under both rules: with the fallback, 10.3 % of accepted calls did not compile, against the base's 7.9 % and a limit of 9.9 %, and without it 9.0 % against 5.5 % and 7.5 %; Opus's and Fable's edits met this guard under both. A breakdown by repository, made after reading the result, puts the excess in one repository: 10 of the 21 new compile failures fall on its two tasks, and without it the change meets the guard under both rules. So the dependence on the matching rule is not specific to weight edits, the round-1 gain of 2.0 points did not recur on this set, and the tool-side change also failed the round-2 guards, on the compile guard alone.

4 Related work and scope

Weight-space programming showed that frontier models can write changes to network weights directly, on small networks and a 0.8-billion-parameter model [1]; we add a deployed coding agent, held-out repositories, live runs and a gradient baseline. ROME writes a human-designed rank-one update where a fact is stored [6], and activation steering changes intermediate activations at inference time [7]. Left to themselves, the editors rebuilt known families of edits, but none tried updates that restrict or penalise changes on preserved calls' activations, such as a null-space constraint [9] or a covariance-weighted key-value edit [10]; with preservation as the binding constraint, these are the obvious next test, though they bound activations, not outputs. Barry [13] fitted a rank-one edit with a linear program that bounds its effect on preserved tokens. On the benchmarks of [1], it matched the frontier model's edit of [1] that stops a model from repeating itself, while changing 8× fewer weights, and a steering vector matched it too. Because a constant push sufficed there, Barry names a behaviour that changes some tokens and leaves others alone as the better test. We add that escape artefacts are such a behaviour: escapes must leave the old text but stay where the file has them, and one editor's candidate broke legitimate escapes in new code. Agent success also depends on its tool interface [8], as our tool-side results in Section 3.5 show.

The study covers one model, one main target, one run per editor, one live run per arm in each setting, and one training seed. The companion writeup adds a second target, keeping the pydantic 1 API in projects pinned to pydantic 1, where E4B writes pydantic 2 names: three agent-written edits cut the share of such wrong-version answers on held-out cases from 0.29 to at most 0.04, and all three failed a preservation check. We chose the target after counting artefacts in the base model's test calls, and the deployed edit after the replay; the replications were planned after both choices, so they test the chosen edit but do not undo the selection. Missing traces and file states reduce the paired samples, and we did not serve an all-zero adapter, so the effect of serving through an adapter is not separated from the effect of the changed weights.

The lesson that carries forward: a tool's tolerance decides what a behaviour fix is worth. Where the tool already repairs a malformed argument, removing the malformation gains nothing on those calls and can lose some; without the repair, the same edits gain. Preservation must therefore be tested apart from the target change, and live behaviour apart from both. Before spending an editor's budget, it pays to measure how much an ideal fix of the target could gain, as the retry checks did.

5 Resources

`weightprog` (Apache-2.0, alpha) packages edit constructions, model instruments, size-matched random controls and comparisons, with MLX and PyTorch backends, for editing Gemma 4 toward a new target from a user's own coding agent. A companion writeup describes it and the data below. On the real E4B its base generation, backend agreement and captured edit effects match the study's outputs; edited generation has run through its MLX backend, and this study's adapters came from its export helper. OOD-C packages the 186 tasks for offline agent runs (SWE-rebench sources, CC-BY-4.0; open-source repositories only; 14.4 GB with snapshots and wheels), with gradability flags. The replay set holds the 320 held-out calls with their prompts and file states. The repository also holds the plans, logs, frozen reports, editor records and a long-form record of every test. A

Kaggle notebook loads the model, applies Fable's edit and shows its effect token by token; it also fits a rank-1 edit in closed form and exports it as a LoRA adapter. Library: github.com/ozgung/weightprog; OOD-C: kaggle.com/datasets/ozgung/g4-oodc; replay set: kaggle.com/datasets/ozgung/g4-weight-edit-replay; repository: github.com/ozgung/gemma4-weight-edit-study; notebook: kaggle.com/code/ozgung/what-a-weight-edit-does-in-gemma-4-e4b.

6 Conclusion

Frontier models wrote gradient-free weight edits that reduced a specific coding-agent behaviour beyond matched controls, and one edit kept that change in the live agent, with thinking off and on. No edit or gradient-trained adapter met our preservation rule under the competition's harness, and no task benefit was detected. Re-scored post hoc without the harness's repair, two edits met the guards and raised acceptance: the measured cost depends on the matching rule as well as the edit.

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
3. van Miltenburg, van der Lee, Krahmer. Preregistering NLP Research. NAACL 2021.
4. Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. NeurIPS 2025.
5. Badertdinov, Nekrashevich, Shevtsov, Golubev. SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale. 2026.
6. Meng, Bau, Andonian, Belinkov. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
7. Turner et al. Steering Language Models With Activation Engineering. arXiv 2023.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
10. Meng, Sharma, Andonian, Belinkov, Bau. Mass-Editing Memory in a Transformer. ICLR 2023.
11. Hewitt, Liang. Designing and Interpreting Probes with Control Tasks. EMNLP 2019.
12. Yan et al. Positive-Congruent Training: Towards Regression-Free Model Updates. CVPR 2021.
13. Barry. Can a Linear Program Do Weight-Space Programming?. Blog post, 2026.